

Learning from Demonstration for Complex Tasks: Vision-Language-Action Models for OARbot Control

Pol del Castillo

2026-08-28

Table of contents

1	LfD for Complex Tasks — VLA Control of OARbot	5
1.1	Project schedule	5
1.2	Initiative board	6
1.2.1	A — Foundations — LfD, VLM/VLA, and the task suite	6
1.2.2	B — Platform & data — OARbot bring- up and demonstrations	7
1.2.3	C — Policy learning — GR00T and HF VLA fine-tuning + evaluation	7
1.3	Item board	8
1.3.1	Stale items	8
1.4	Where to go from here	8
I	Background	9
2	Background	10
3	Example section	11
II	Methods	12
4	Methods	13
III	Operations	14
5	Operations	15
6	Fall 2026 Roadmap — experiments to December	16
6.1	Research questions	16
6.2	Compute and platform allocation	17

6.3	Month-by-month plan	17
6.3.1	September — bring-up, task suite, literature (epics B1, A2, A1)	17
6.3.2	October — dataset at scale + first baseline (epics B2, C2)	19
6.3.3	November — GR00T + full evaluation (epics C1, C3)	20
6.3.4	December — finish, freeze, outline (epics C3, D)	20
6.3.5	January–March 2027	21
6.4	Quantifiables — what gets measured and reported	21
6.4.1	Decision thresholds — what “achieved” means	21
6.4.2	Primary metrics (every policy, every task)	22
6.4.3	Secondary metrics	22
6.4.4	Generalization tiers (RQ2)	23
6.4.5	Failure taxonomy	23
6.4.6	Practicality cost sheet (RQ3)	23
6.4.7	Deliverables checklist (results chapter) . .	24
6.5	Literature map	24
6.6	Open-source stack	26
6.7	Using AI research assistants	27
6.8	Risk register	27
6.9	Cadence	28
7	OARbot platform notes — findings & day-1 checklist	29
7.1	Findings — how the OARbot communicates . . .	29
7.1.1	Network and middleware	29
7.1.2	Hardware (per robot)	30
7.1.3	The arm interface (kinova-ros2)	30
7.1.4	Control-rate design (why the policy loop is ~10 Hz)	31
7.1.5	Pipeline decisions locked in so far	32
7.2	Day-1 checklist — completed 2026-08-27	33
8	Lab notebook	35
8.1	2026-08-27 — Day 1 with the robot	35
8.1.1	The goal	35
8.1.2	What happened	35
8.1.3	The numbers that shape what comes next	36
8.1.4	Bottom line	37

9	SOP-01: Example procedure	38
9.1	Purpose	38
9.2	Prerequisites	38
9.3	Verification	38
9.4	Change history	38
10	Future work	39
	Appendices	40
A	Weekly updates	40
B	Initiatives	41
C	Archive	42
C.1	Buckets	42
D	External archive index	43
D.1	Always-external (vendor + heavy media)	43
D.2	Oversize files skipped from buckets (> 25 MB)	43

1 LfD for Complex Tasks — VLA Control of OARbot

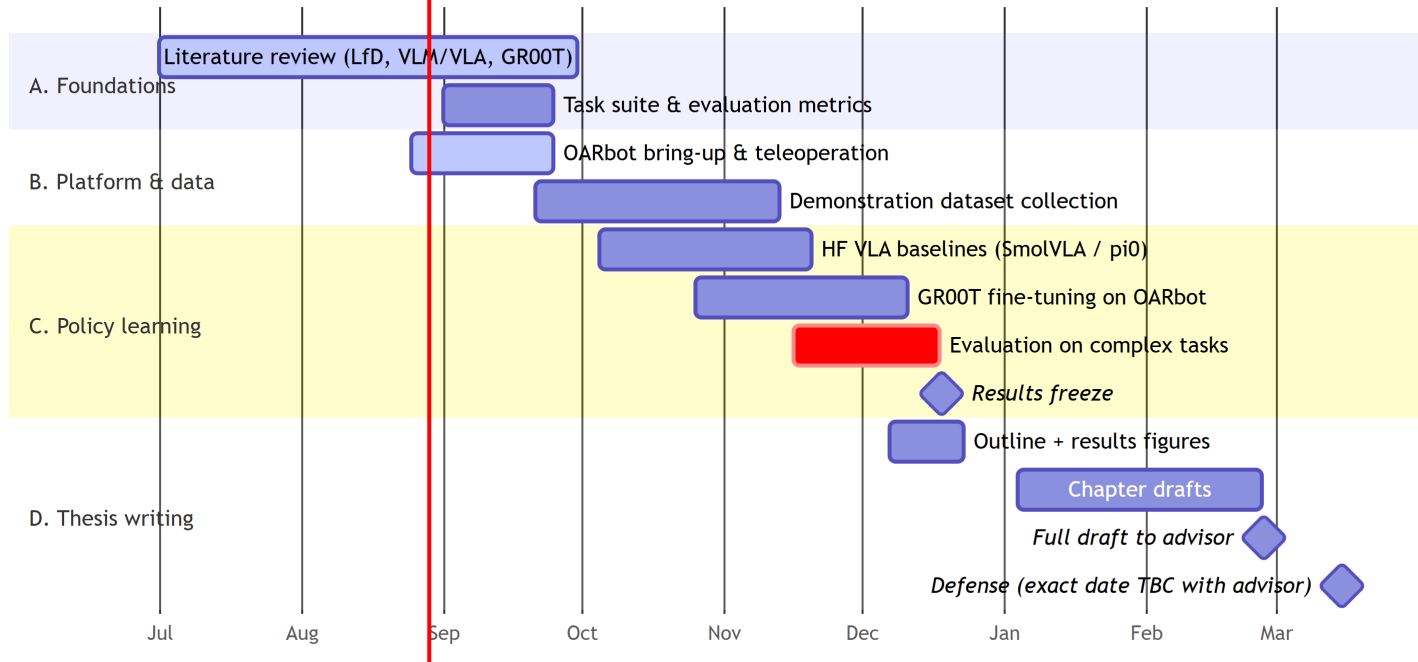
This site has two parts. The **thesis** is the prose in [Background](#), [Methods](#), [Operations](#), and [Future Work](#), that's what gets bound. The rest of the site is the **project-management surface** the thesis runs on: a top-level Gantt, the initiative board below, a weekly-update log, and an archive of source material.

TODO: replace this paragraph with the abstract once the thesis is drafted.

1.1 Project schedule

Top-level Gantt covering the work categories (need to confirm with advisor). The Mermaid block below is the single source of truth for the schedule. Editing this block is possible when the schedule changes; the per-WU frozen PNGs (`updates/WU-*/media/gantt.png`) regenerate automatically from it via `scripts/render/build_gantt.py`. Add a bar here whenever a new epic is scaffolded (`.\thesis.ps1 new epic ...`).

Thesis schedule — experiments to Dec 2026, writing Jan-Feb, defense Mar 2027



1.2 Initiative board

The Gantt's bars are *epics*. Each epic has a README under [initiatives/](#) with a user/need/benefit brief, related-epics graph, and an items listing. Everything below is generated from README front matter by `scripts/render/build_board.py` (a pre-render hook), edit the front matter, never these tables. Schedule emphasis (crit bars) lives in the Mermaid Gantt above.

1.2.1 A — Foundations — LfD, VLM/VLA, and the task suite

Gantt	Epic	Status	Owner
A1	Literature review: LfD, VLM/VLA, GR00T	in-progress	Pol del Castillo
A2	Task suite & evaluation metrics	open	Pol del Castillo

1.2.2 B — Platform & data — OARbot bring-up and demonstrations

Gantt	Epic	Status	Owner
B1	OARbot bring-up & teleoperation	open	Pol del Castillo
B2	Demonstration dataset collection	open	Pol del Castillo

1.2.3 C — Policy learning — GR00T and HF VLA fine-tuning + evaluation

Gantt	Epic	Status	Owner
C1	GR00T fine-tuning on OARbot	open	Pol del Castillo
C2	HF VLA baselines (SmolVLA / pi0)	open	Pol del Castillo
C3	Evaluation on complex tasks	open	Pol del Castillo

1.3 Item board

Open 1

A1-EX-001

Which VLA backbones fit OARbot and the available compute?

A1 · Pol del Castillo

In progress 0

Blocked 0

Done 0

1.3.1 Stale items

in-progress items with no commit in 3+ weeks.

None — every in-progress item has recent activity.

1.4 Where to go from here

- **Reading the thesis** — start with [Background](#) and use the sidebar.
- **Reviewing weekly progress** — see the [weekly updates](#) appendix (newest first).
- **Browsing the project tree** — see the [initiatives](#) appendix; each initiative lists its epics, each epic lists its items + status log.
- **Looking up source material** — see the [archive](#) appendix, with an [external index](#) for files too large to carry in git.

Part I

Background

2 Background

TODO: Part intro — why this research area matters and what question the thesis answers.

3 Example section

TODO: First content section. Chapter subsection files follow the `CH-<N><x>-<slug>.qmd` naming convention and are registered in `_quarto.yml` under `book.chapters`. Cite sources with `@key` against `references.bib`.

Part II

Methods

4 Methods

TODO: Part intro — equipment, experimental setup, and how evidence flows from the lab into the **initiatives/** tree (every experiment in the PM layer that produces thesis-grade results gets written up here).

Part III

Operations

5 Operations

TODO: Part intro for operating procedures. SOPs live in `sops/` and render in three audience variants via Quarto profiles (`--profile sop-detailed | sop-checklist | sop-walkthrough`). Each SOP carries a formal revision block (rendered by the shortcode) plus an auto-generated git change history.

6 Fall 2026 Roadmap — experiments to December

Working plan from today (Aug 20) to the results in December/January. The [top-level Gantt](#) is the source of truth for dates; this page holds the reasoning behind them: research questions, week-level milestones, the quantifiables spec (metrics, generalization tiers, failure taxonomy), the literature map, the open-source stack, and how AI research assistants are used. Revise it whenever the plan changes, it becomes the skeleton of the Methods chapter and the defense timeline slide.

6.1 Research questions

The thesis claim is built around three questions. Every experiment should answer one of them.

RQ1 — Adaptation. Can current open vision-language-action (VLA) foundation models (NVIDIA GR00T N1.5, SmolVLA, optionally 0) be adapted to a *new embodiment*, the OARbot (Clearpath Dingo omni base + Kinova Jaco Gen2 6-DOF arm) with a modest demonstration dataset collected in the lab?

RQ2 — Comparison & generalization. On a fixed task suite, how do the fine-tuned VLAs compare against each other *and against a from-scratch imitation baseline (ACT) trained only on our demonstrations*, both on in-distribution success rate and on how performance degrades across generalization tiers (unseen poses, distractors, novel object instances, paraphrased instructions)? The baseline is what makes generalization attributable: if the VLAs degrade less than ACT out of distribution, that gap *is* the measured value of internet-scale pretraining.

RQ3 — Practicality. What are the real compute and data costs, fine-tuning wall-clock on a DGX Spark, demos-per-task needed, on-robot inference rates, and what does that imply for small-lab replication?

RQ3 is the differentiator: most VLA papers report results on the authors’ hardware fleet; a careful research of what it takes to adapt these models on one robot with two lab machines is a new contribution.

6.2 Compute and platform allocation

Machine	Role
DGX Spark (GB10, 128 GB unified memory)	All fine-tuning: GR00T N1.5 (NVIDIA ships a DGX Spark fine-tuning playbook for it), SmolVLA full fine-tune, 0 LoRA if attempted
RTX workstation	Robot-side: teleop, data recording, policy inference at eval time, dataset visualization
OARbot helper laptop + Dingo onboard PC	ROS 2 Jazzy stack, drivers, cameras (per the OARbot docs)

Important to keep training and robot operation on separate machines from day one so data collection never competes with training runs.

6.3 Month-by-month plan

6.3.1 September — bring-up, task suite, literature (epics B1, A2, A1)

- **Week of Sep 1:** Full teleop chain on one OARbot: drive the base, command the arm (SpaceMouse), stream both

Azure Kinects. Verify E-stop. Note which sensors are actually usable (docs already flag the UWB units as inoperable).

- **Week of Sep 8:** Record demonstrations into **LeRobot-Dataset format**: joint states + (base velocity, future proof) + camera frames, replay a recorded episode open-loop on the robot to prove the pipeline is lossless. This replay test is very important to corroborate the strategy works.
- **Week of Sep 15:** Freeze the **task suite** (epic A2): 3 tasks in increasing difficulty, e.g. (T1) tabletop pick-and-place, arm only; (T2) fetch and navigate to a table, grasp, return; (T3) a true mobile-manipulation task with base+arm coordination (open a door, clear a table). Define per-task success criteria, the staged **progress rubric** (reach / grasp / transport / place), the in-distribution randomization grid, and the **generalization tiers G1–G4** (see the quantifiabiles spec below) *before* collecting data. Confirm the suite with the advisor. Also start the **action-space decision log**: every embodiment-design choice (joint deltas vs. absolutes, base twist representation, chunk length, control rates, normalization) gets a dated entry with the reasoning, this log becomes the Methods section on adapting VLA action heads to a mobile manipulator.
- **Week of Sep 22:** Pilot data collection: ~10 episodes per task, train a quick SmolVLA sanity run on T1, fix whatever breaks.
- **Gate 1 (Sep 26):** teleop + record + replay all work, task suite signed off. *Fallback if late: drop T3 base motion and run an arm-centric thesis, decide here.*

6.3.1.1 Data-pipeline decisions (locked 2026-08-25)

- Teleop is **Cartesian** (SpaceMouse → EE twist → Kinova IK); training actions are **joint positions** derived from achieved states: $\text{action}(t) = \text{joint angles at } t + \Delta$. Teleop space and training space are decoupled by this labeling — the raw SpaceMouse commands are never used as labels.
- Recording = **ros2 bag record** on the **helper laptop** (compressed image topics, one auto-numbered bag per

episode), then an offline **bag** $\rightarrow$ **LeRobotDataset converter**: uniform rate, nearest-frame alignment, the frozen 10-D schema, $t+\Delta$ action labels, instruction string per episode. Raw bags are kept forever as source of truth — every labeling choice stays reversible without re-collection.

- **Wrist camera mounted before any kept demos** — observations are locked in at recording time; actions can be re-derived, images cannot.

6.3.1.2 Week-1 verification checklist (at the robot)

1. `ros2 topic list` + `ros2 topic hz` during teleop: arm JointState topic (finger joints included?), camera topics, `/odom`, raw teleop command topic if exposed. Note exact names and rates.
2. **Angular control** — does the lab stack expose joint position/velocity *streaming* to the Gen2? Teleop only proves the Cartesian path; policies need the joint path. Biggest unknown on the platform. (*Resolved 2026-08-26: the kinova-ros2 driver streams `in/joint_velocity` at exactly 100 Hz — see the [platform notes](#).*)
3. Gripper commands land on a ROS topic and appear in recordings.
4. Round-trip latency: camera $\rightarrow$ RTX box $\rightarrow$ command back to the helper laptop (decides wireless inference viability; chunk $\sim 3\times$ latency).
5. Clock sync (chrony) across Dingo PC / helper laptop / NUC / RTX box — required for correct chunk alignment at deployment.
6. End-to-end proof: record one episode $\rightarrow$ convert $\rightarrow$ **replay** it on the arm via angular control.

6.3.2 October — dataset at scale + first baseline (epics B2, C2)

- **Weeks of Oct 5 / Oct 12**: Main collection push, target **50–100 episodes per task** (ACT/SmolVLA-class results are routinely reported around 50 demos/task; more helps T3). Interleave tasks and vary object poses per the

randomization grid. Nightly QC script: episode counts, joint-limit outliers, dropped frames.

- **Week of Oct 19:** SmolVLA fine-tuned per-task on the Spark, and **ACT trained from scratch on the identical data** — it trains in hours and rides along on the same LeRobot dataset, giving the no-pretraining reference point for RQ2. First closed-loop rollouts on the robot.
- **Week of Oct 26:** Start GR00T N1.5 fine-tuning (epic C1) — the new-embodiment head needs the most debugging lead time. First structured mini-eval (10 trials/task) of SmolVLA.
- **Gate 2 (Oct 31):** SmolVLA achieves non-zero success on T1 with a documented eval protocol. *If zero: freeze collection, debug the data/normalization/camera pipeline first — more bad data won't help.*

6.3.3 November — GR00T + full evaluation (epics C1, C3)

- **Weeks of Nov 2 / Nov 9:** GR00T N1.5 tuned across tasks; top-up data collection where QC shows gaps; 0 (LoRA via LeRobot/openpi) only if both main models are stable.
- **Week of Nov 16:** Full evaluation begins (epic C3): **20 trials/task/policy** on the shared in-distribution grid, everything in the quantifiables spec logged per trial, video of every trial. Budget realistically — 3 tasks $\times$ 4 policies (ACT, SmolVLA, GR00T, 0-if-attempted) $\times$ 20 trials at ~ 3 min/trial is ~ 12 robot-hours *per full round*, plus resets.
- **Gate 3 (Nov 27):** Decide final comparison set (ACT + 2 or 3 VLAs) and confirm the **data-scale ablation** (retrain at 25/50/100 demos on one task) fits the calendar.

6.3.4 December — finish, freeze, outline (epics C3, D)

- **Weeks of Dec 1 / Dec 7:** Complete in-distribution eval rounds + data-scale ablation; **generalization round** — tiers G1–G4 on each model's best checkpoint only (10 trials/tier, see quantifiables spec).

- **Week of Dec 14:** Re-runs and holes only. **Results freeze Dec 18** — after this date, no new experiments, only analysis.
- **Week of Dec 21:** All results tables and figures generated by script from logged data; chapter outline drafted in `chapters/`.

6.3.5 January–March 2027

Chapter drafts Jan 4 – Feb 26 (Background and Methods first — most of Methods should fall out of this page and the epic READMEs), full draft to advisor end of February, defense mid-March (confirm the exact date and committee logistics with the advisor **in September**, not January).

6.4 Quantifiables — what gets measured and reported

What the results chapter must contain, modeled on what this class of papers (ACT, Diffusion Policy, OpenVLA, 0, SmolVLA, GR00T, Mobile ALOHA) actually reports. Frozen in epic A2 before data collection; every eval trial logs a row with all of these fields plus a video path.

6.4.1 Decision thresholds — what “achieved” means

Targets are pre-registered thresholds, not promises. A miss, analyzed honestly, is still a result.

- **Platform validated:** 1 policy reaches **70% success on T1** in-distribution.
- **Pretraining transfer confirmed:** the VLAs beat from-scratch ACT on the generalization tiers by **15 percentage points** average. A null or negative gap = “pretraining did not pay off at this data scale on this embodiment”, a publishable negative result.
- **Practicality documented:** complete cost sheet (below) for every model, no gaps.

6.4.2 Primary metrics (every policy, every task)

Metric	Definition	Reported as
Success rate	Binary per trial against the frozen rubric; human intervention = failure	% with 95% Wilson CI, n = 20
Progress score	Staged credit: reach 0.25 / grasp 0.5 / transport 0.75 / place 1.0 (per-task rubric)	Mean 0–1; separates “never grasps” from “drops at the end” when success rates are low
Time to completion	Successful trials only, start signal → success condition	Median [IQR] seconds

6.4.3 Secondary metrics

Metric	Definition	Reported as
Inference latency	Camera frame in → action chunk out, on the RTX workstation	ms per chunk + achieved control rate (Hz)
Generalization gap	In-distribution success minus per-tier success	Percentage points, per tier per policy
Data efficiency	Success rate at 25 / 50 / 100 training demos (one task, all policies)	Curve — the small-lab money figure

6.4.4 Generalization tiers (RQ2)

Evaluated on each model’s **best checkpoint only**, 10 trials/tier, selection rules frozen in A2:

- **G0** — in-distribution: objects and poses from the training grid.
- **G1** — **spatial**: object poses outside the training grid.
- **G2** — **visual**: distractor objects + lighting change.
- **G3** — **object**: novel instance of the same category (different mug, different box).
- **G4** — **language**: paraphrased instruction, same task (“pick up the cup” → “grab the mug”). VLA-only tier — ACT has no language input, which is itself part of the comparison story.

6.4.5 Failure taxonomy

Every failed trial gets a **first-failure label** from this closed list, logged at eval time (10 seconds of effort) with the video path:

perception/grounding (approached wrong object) · navigation/base-pose (stopped out of reach) · grasp (contact but slip/miss) · manipulation-trajectory (jerky/unsafe motion, joint limit) · instruction (did the wrong task) · stall/timeout · hardware (excluded and retried, not counted).

Rendered as stacked bars per policy × task — this figure explains *why* the success rates differ and is the backbone of the analysis chapter.

6.4.6 Practicality cost sheet (RQ3)

Per model: fine-tuning wall-clock hours and peak memory on the DGX Spark, trainable parameter count (LoRA vs. full), GPU-hours per training run, on-robot inference hardware requirements. Plus data cost: minutes per collected episode, total collection hours, retake rate.

6.4.7 Deliverables checklist (results chapter)

- **T1** Main results: success $\pm$ CI, task $\times$ policy, in-distribution
- **T2** Generalization matrix: policy $\times$ tier
- **T3** Efficiency: latency, control rate, completion time
- **T4** Cost sheet (RQ3)
- **F1** Data-scale curve · **F2** Failure-mode stacked bars · **F3** Qualitative rollout film-strips (best + worst per policy)

Protocol hygiene: the randomization grid and rubrics are frozen before collection; checkpoint selection criteria defined before eval; all runs reported (no cherry-picked seeds); every trial has video.

6.5 Literature map

Organized by the Background chapter’s likely sections. Maintain citations in Zotero and export to `references.bib` (keep BibTeX keys stable). Verify every reference on arXiv/Semantic Scholar before it enters the bib file — never cite from an LLM’s memory.

1. Learning from demonstration — foundations

- Argall, Chernova, Veloso, Browning (2009), *A survey of robot learning from demonstration* — the canonical survey to anchor the chapter.
- Pomerleau (1988), ALVINN — behavioral cloning origin.
- Ross, Gordon, Bagnell (2011), DAgger — compounding-error argument every IL thesis needs.

2. Modern imitation learning architectures

- Zhao et al. (RSS 2023), ACT / ALOHA — action chunking; the direct ancestor of today’s VLA action heads.
- Chi et al. (RSS 2023), Diffusion Policy — generative action heads.
- Lipman et al. (ICLR 2023), Flow Matching — theory behind 0-style and GR00T-style flow action heads.

- Fu, Zhao, Finn (2024), Mobile ALOHA — the closest published setup to this thesis: whole-body IL on a mobile manipulator.

3. Vision-language-action models

- Brohan et al., RT-1 (2022) and RT-2 (2023) — the VLA lineage.
- Open X-Embodiment collaboration (2023/24) — cross-embodiment pretraining; why adapting a generalist to OARbot is plausible at all.
- Octo Model Team (2024) — open generalist policy.
- Kim et al. (2024), OpenVLA — open 7B VLA + fine-tuning recipes (see also the OpenVLA-OFT follow-up on efficient fine-tuning).
- Black et al. (2024), 0; Physical Intelligence (2025), 0.5 — flow matching VLA, open weights via openpi.
- Hugging Face (2025), SmolVLA — small (~450M) VLA designed for exactly this compute class; likely the workhorse baseline.
- NVIDIA (2025), GR00T N1 paper (+ N1.5 release notes) — the primary model; its dual-system architecture and new-embodiment fine-tuning story are directly load-bearing.
- Google DeepMind (2025), Gemini Robotics — closed, but frames the state of the art in Background.

4. Mobile manipulation and assistive context

- TidyBot (2023), HomeRobot/OVMM (2023), BEHAVIOR-1K (2023) — mobile-manipulation task framing and benchmarks.
- RPI’s own OARbot publications — get the list from the advisor; the assistive-robotics motivation should cite the lab’s line of work.

5. Datasets and evaluation methodology

- DROID (2024), BridgeData V2 (2023), AgiBot World (2025) — large-scale teleop dataset practice worth imitating at small scale.
- LIBERO (2023) — task-suite design and eval discipline.

- SimplerEnv — Li et al. (2024), *Evaluating Real-World Robot Manipulation Policies in Simulation* — the real-vs-sim eval gap.
- Recent work on statistically sound robot policy evaluation (trial counts, confidence intervals, A/B protocols) — good target for a deep-research sweep, see below.

6.6 Open-source stack

Resource	Use here
LeRobot	Dataset format, teleop recording, SmolVLA + 0 (PyTorch) training/eval — the backbone of B2/C2
Isaac GR00T	GR00T N1.5 checkpoints + new-embodiment fine-tuning scripts; check NVIDIA’s DGX Spark playbook
openpi	0/ 0.5 reference implementation (JAX) if the LeRobot port falls short
oarbots repo	Lab’s ROS 2 stack, drivers, teleop — extend, don’t rewrite
Hugging Face Hub	Host the OARbot demonstration dataset (private repo) + fine-tuned checkpoints; free provenance and versioning
Weights & Biases (or TensorBoard)	Training curves + eval run tracking, linked from weekly updates
Zotero → <code>references.bib</code>	Single bibliography pipeline into Quarto

Stretch only (explicitly out of scope unless everything lands early): Isaac Lab / SimplerEnv-style sim twin of a task for cheap eval — you have Isaac Sim experience from the SO-101

project, but a sim twin is a second thesis; park it in Future Work.

6.7 Using AI research assistants

Division of labor — each tool where it is strongest:

- **Gemini Deep Research** breadth: one report per Background section above (“annotated bibliography of VLA fine-tuning on novel embodiments 2023–2026, with arXiv links”). Also the eval-methodology sweep in §5.
- **ChatGPT (deep research)** cross-check Gemini’s reports; protocol questions (“how many trials for a detectable 15-point success-rate difference between two policies?”).
- **Claude Code** everything in-repo: Keeping this PM surface honest and clean.

Standing rule: LLM output is a *lead*, never a *citation*. Every claim that survives into the thesis gets verified against the primary source, and every reference is checked on arXiv before entering `references.bib`.

6.8 Risk register

Risk	Likelihood	Mitigation
Bring-up eats October (drivers, arm, cameras)	High	Gate 1 fallback: arm-only task suite, decided Sep 26
Demo data too little/noisy for T3	Medium	Tiered tasks — T1/T2 alone still answer RQ1–RQ3; data-scale ablation turns “not enough data” into a finding

Risk	Likelihood	Mitigation
GR00T new-embodiment head underperforms	Medium	SmolVLA is the guaranteed baseline; a negative GR00T result, carefully analyzed, is still a thesis result
Eval time underestimated	High	Protocol fixed in Sept; 20 trials budgeted on the calendar, not squeezed in
Generalization round balloons eval time	Medium	Tiers only on best checkpoints, 10 trials/tier; if squeezed, run G1-G4 on T1+T2 only
Two-robot temptation (multi-robot experiments)	Medium	One OARbot is the experiment platform; the second is spare parts / backup until the freeze
Writing starts late	Medium	Section D outline bar in December; Methods accretes from epic READMEs all semester

6.9 Cadence

Weekly update (`.\thesis.ps1 new wu`) every Friday: progress, frozen Gantt snapshot, blockers, questions for Prof. Wen. The board and Gantt on the [home page](#) stay the single source of truth.

7 OARbot platform notes — findings & day-1 checklist

Compressed findings from digesting the OARbot documentation ([oarbots repo](#) · [docs site](#) · [kinova-ros2](#)) before first contact with the robot, plus the reconnaissance checklist for the first lab session. Feeds epic [B1](#); the reasoning behind the pipeline lives in the [Fall 2026 roadmap](#). Update this page as the topic dump and rate measurements replace assumptions with numbers.

7.1 Findings — how the OARbot communicates

7.1.1 Network and middleware

- ROS 2 **Jazzy** with **CycloneDDS** (`RMW_IMPLEMENTATION=rmw_cyclonedds_cpp`) on the lab Wi-Fi CII8218-oarbots. *Not* Fast DDS — keep this machine’s env separate from the Isaac Sim / SO-101 setup.
- Discovery is peer-to-peer (no master): any machine on the Wi-Fi with domain ID 0 sees everything. Connectivity check: `ros2 topic list`.
- **Domain layout:** each robot runs internally in its own ROS domain — Blue = 2, Silver = 1 — and a **domain bridge** re-publishes both onto the shared domain 0 with `/oarbot_blue/...` and `/oarbot_silver/...` prefixes. The bridge is bidirectional: sensor streams out, commands in.
- Multi-network trouble → pin the interface in `~/.ros/cyclonedds.xml` and set `CYCLONEDDS_URI`.

7.1.2 Hardware (per robot)

Component	Detail
Base	Clearpath Dingo DO-100, mecanum (omnidirectional), onboard PC runs its ROS nodes
Arm	Kinova Jaco Gen2 6-DOF, 3-finger (j2n6s300), USB to helper laptop
Helper laptop	Lenovo ThinkPad on the base; runs <code>kinova_driver</code> + Kinect node, auto-launches at boot
Cameras	Azure Kinect on each arm (RGB-D, 30 fps max); overhead Kinect on a NUC does ArUco pose estimation (Blue tag 0, Silver tag 4)
F/T sensor	Bota Rokubi Gen0 at the wrist — enables admittance (kinesthetic) mode
Teleop	Surface Go 2 tablet: SpaceMouse Compact + deadman switch + E-stop; GUI via <code>tablet_launch.sh</code>
Inoperable	UWB units (DecaWave MDEK1001) — ignore

Boot order is strict (nodes fail to launch otherwise): batteries → Dingo → arm (hold home) → helper laptop → NUC (replug Kinect USB) → tablet → disengage E-stop → launch GUI.

7.1.3 The arm interface (kinova-ros2)

Topic names below are the stock driver names; on the lab network they appear under the bridge prefix (exact names TBC from the topic dump).

Direction	Topic	Type / unit	Notes
out	...driver/out/joint_states	JointStates, rad	JointState, proprioception
out	...driver/out/tool_pose	pose	
out	...driver/out/tool_wrench	wrench	
out	...driver/out/finger_position	finger_position	
in	...driver/in/joint_velocities	JointVelocities, deg/s	at exactly 100 Hz
in	...driver/in/cartesian_msgs/Pose	Pose, 100 Hz	100 Hz requirement

Services: `home_arm`, `start/stop`, `start_force_control/stop_force_control` (the tablet’s admittance button), `set_zero_torques`. Actions: joint trajectory, pose goal, fingers (percent/mm/turns).

Three load-bearing facts from the driver README:

1. **100 Hz is exact, not a minimum.** The arm’s DSP loops every 10 ms; slower publishing fails to reach the commanded velocity, faster does nothing. Any policy or teleop source must run through a 100 Hz command repeater.
2. **Publishing stops motion stops.** The velocity stream is a built-in deadman — killing the publisher halts the arm.
3. **Unit trap:** commands in deg/s, joint states in rad. Convert in exactly one place in the pipeline; log the choice in the action-space decision log.

7.1.4 Control-rate design (why the policy loop is ~10 Hz)

Two decoupled rates, connected by action chunking: the *decision* loop (model inference) at ~10 Hz, the *command* stream at 100 Hz fed by a repeater executing the current action chunk. The decision rate is bounded by the camera (30 fps ceiling), model latency (ACT ~ms, but SmolVLA/GR00T/0 are 50–200 ms per

chunk on the RTX workstation), and end-to-end Wi-Fi latency. Published VLAs run 1–10 Hz re-planning with high-rate chunk execution (0: 50 Hz chunks, ~2 Hz model calls). Achieved latency/control rate is an RQ3 metric — measured per model, not assumed.

7.1.5 Pipeline decisions locked in so far

- **Bag-first recording:** `ros2 bag record` (one bag per episode, keypress wrapper) is the recorder node — record generously (joints, RGB, depth, wrench, odom, both command topics, `/tf`, `camera_info`). Converter script resamples offline to a fixed policy-rate clock and emits LeRobot-Dataset; converter bugs cost a re-run, not a re-collection.
- **Action labels come from achieved states, not raw teleop** (locked 2026-08-25): teleop stays Cartesian (SpaceMouse $\rightarrow$ EE twist $\rightarrow$ Kinova IK), while training actions are joint positions at $t+\Delta$ derived from the recorded joint states — teleop space and training space are decoupled. Details in the roadmap’s [data-pipeline decisions](#).
- **Replay test is rung 0 of the model ladder:** republish a recorded episode’s commands at original timing; the arm must reproduce the demo before any learning starts. Ladder: replay $\rightarrow$ ACT $\rightarrow$ SmolVLA $\rightarrow$ GR00T N1.5 $\rightarrow$ (0).
- **Inference node mirrors the recorder:** same topics in, same preprocessing as training (normalization stats ship with the checkpoint), chunk execution + 100 Hz repeater out, keyboard deadman publishing zeros, human at the E-stop. Dry-run the plumbing with a dummy policy (zeros / replayed chunk) before the first trained model.
- **Admittance mode is a candidate second demo modality** (kinesthetic guiding via the F/T sensor) — bake off against SpaceMouse teleop in the pilot week; docs warn calibration can be flaky, keep E-stop close.

7.2 Day-1 checklist — completed 2026-08-27

All exit criteria met. Plain-language account in the [lab notebook](#); raw dumps and command captures in WSL `~/oarbot_recon_2026-08-27/`.

1. **Env sanity** — plus the unplanned fixes that made discovery work from WSL at all: mirrored networking, lab Wi-Fi marked Private, and the decisive Hyper-V firewall rule (inbound UDP 7400–7500 into WSL); `~/ros/cyclonedds-oarbot.xml` (interface pin
 - unicast peers) wired into `.bashrc`.
2. **Inventory dumps** — 49 topics, 25 nodes, 176 services, 8 actions (only Blue was powered).
3. **Rates** — combined `/oarbot_blue/joint_states` (16 joints: 4 wheels + 6 arm + 6 fingers, standard `JointState`) at ~9.7 Hz; Kinect 1920×1080 at ~14.6 fps internally, raw RGB arriving at only ~3.4 Hz over Wi-Fi.
4. **Camera transport** — no `/compressed` topics exist; Wi-Fi drops ~75 % of raw frames → record on the helper laptop, as the Aug 25 data-pipeline decision anticipated.
5. **Command topics** — identified in `tablet_gui` source and confirmed by live capture (1,073 messages): arm = `PoseVelocity` on `in/cartesian_velocity` at ~119 Hz while the deadman is held; fingers = `PoseVelocityWithFingers` (closure %) event-based on `in/cartesian_velocity_with_fingers`; base = `TwistStamped` on `dingo/cmd_vel`. Quirk: Silver’s x-y axes are flipped in the tablet source. Reading these types required building `kinova_msgs` locally (deactivate conda first — CMake prefers the `CONDA_PREFIX` Python).
6. **Clock check** — laptop 1 s ahead of robot stamps. Harmless for recording (helper laptop = single clock for camera + joints); `chrony` required before live deployment.
7. **Command authority** — 150 `PoseVelocity` messages at 100 Hz (`twist_linear_z: 0.02`) sent from the laptop raised the arm ~3 cm; motion stopped when publishing stopped. External control confirmed — the exact channel a policy will use.

Next per the [roadmap](#): the recording pipeline on the helper laptop (`ros2 bag record`, one bag per episode).

8 Lab notebook

Session-by-session record of hands-on work with the robots, newest first — one dated entry per session, written so both an advisor and a non-engineer can follow it. The technical reference behind these entries is the [platform notes](#); formal weekly summaries live in the [weekly updates](#).

8.1 2026-08-27 — Day 1 with the robot

8.1.1 The goal

Before a robot can learn anything, my laptop and the robot have to talk to each other. Today’s job was to establish that conversation: hear everything the robot senses, and prove my laptop can tell the robot what to do — the two pipes that all the thesis work will flow through.

8.1.2 What happened

Getting connected took detective work. The robot speaks over the lab’s Wi-Fi, but my laptop runs its robot software inside a Linux compartment (WSL), and Windows was silently guarding that compartment’s door: security rules dropped the robot’s replies before they arrived. Everything *looked* configured correctly and nothing worked — until we found the right firewall gate to open. This is now fixed permanently; the laptop connects on its own from here on.

Then the robot came through loud and clear. We catalogued everything it broadcasts — 49 data channels — and watched its joint angles stream live in a terminal while I drove the arm with the teleoperation tablet.

We caught the tablet’s commands mid-flight. While I teleoperated, my laptop recorded over a thousand of the exact command messages the tablet sends the arm. This matters because a learned policy will speak through this same channel — now we know its precise format: six speed values (move/turn in each direction) plus one gripper number. Seven numbers, a hundred times a second. That is the language the robot is commanded in.

And the finale: my laptop moved the arm. A two-second command stream sent from my keyboard raised the arm three centimetres, smoothly, and it stopped by itself — with a hand on the emergency stop the whole time. External control: proven.

8.1.3 The numbers that shape what comes next

Measurement	Value	Consequence
Joint angle updates	~10 per second	matches the planned learning rate
Camera (on the robot)	15 images/sec, full HD	healthy at the source
Camera (over Wi-Fi)	only ~3.4 images/sec arrive	Wi-Fi can’t carry raw video → demonstrations must be recorded on the robot’s own computer , exactly as planned
Tablet command rate	~119 messages/sec	the arm wants 100/sec — confirmed feasible

Clock difference laptop robot	~1 second	harmless for recording (one clock does it all); must be synchronized before live deployment
------------------------------------	-----------	---

8.1.4 Bottom line

Both pipes work: robot $\rightarrow$ laptop (all senses readable) and laptop $\rightarrow$ robot (commands obeyed). The day-1 checklist closed with every box ticked, and the next step is the recording pipeline: capturing human demonstrations on the robot's own computer in the format the learning software expects.

9 SOP-01: Example procedure

9.1 Purpose

TODO: One sentence on what this procedure accomplishes and when it applies. (This SOP exists so the template renders a working example of the revision block + profiles + change history — replace it with your first real procedure.)

9.2 Prerequisites

TODO

9.3 Verification

TODO

9.4 Change history

Owner: TODO Owner

Date	Author	Change
2026-06-12	AmmarBarbee17	Initial commit: Quarto thesis + project-management template

10 Future work

TODO: What the thesis opens up but does not close.
Future-shell epics in the **initiatives**/ tree are good
source material here.

A Weekly updates

TODO: Weekly status slides shared with the advisor. Each entry is a standalone revealjs deck. Click any card to open the slides; reveal.js navigation (arrow keys, ESC for overview) applies.

B Initiatives

TODO: Initiatives are the top-level work categories that mirror the sections of the project Gantt. Each initiative groups its epics; each epic groups its experiments, issues, and features.

C Archive

Historical source-of-truth content imported from outside the repo (cloud drives, old proposals, vendor docs). **Large vendor binaries** and pre-hosted media already represented elsewhere are catalogued in [external-index.qmd](#) rather than copied.

C.1 Buckets

Buckets are filesystem dumps under `archive/`. There is no per-bucket rendered page. Use a local clone to browse contents; this table is the inventory. Files over ~25 MB get an external-index entry instead of a copy (git-lfs carries the rest — see `.gitattributes`).

Bucket	Description	Files
<i>(none yet)</i>	TODO: add a row per imported bucket	—

See [external-index.qmd](#) for files that stay at their original cloud paths.

D External archive index

Files referenced by the archive but **not copied into the repo** (too large for git-lfs to carry efficiently, or already represented elsewhere). They remain at their original cloud paths — record the absolute path so future-you can find them.

D.1 Always-external (vendor + heavy media)

What	Location
<i>(none yet)</i>	TODO: e.g. C:\Users\you\OneDrive – University\research\vendor- installers

D.2 Oversize files skipped from buckets (> 25 MB)

File	Size	Source
<i>(none yet)</i>	—	—